

✓ Reporte de ejercicios

Introducción a Python

Modelos estadísticos

✓ Contenido

- [Ejercicio 1.7: HTML y CSS](#)
- [Ejercicio 1.8: Tipos de datos y operadores](#)
- [Ejercicio 1.9: Estructuras de datos](#)
- [Ejercicio 1.10: Estructuras de control](#)
- [Ejercicio 1.11: Numpy](#)
- [Ejercicio 1.12: Pandas](#)
- [Ejercicio 1.13: Matplotlib](#)
- [Ejercicio 1.14: SQL](#)

✓ Ejercicio 1.7: HTML y CSS

Crea una tarjeta de presentación que incluya tu nombre, saludo, tus estudios universitarios, algo acerca de ti y los lenguajes de programación que conoces. Cambia el fondo de la celda y el color del texto, además incluye emoticones e insignias.

Tarjeta de presentación

¡Hola, soy Héctor Morales Osorio! 🖐️

Estudié Acturaria en la FES-Acatlán.

Me gusta practicar deportes de contacto.

Los lenguajes de programación que utilizo son:

PYTHON

[Volver al contenido](#)

✓ Ejercicio 1.8: Tipos de datos y operadores

Dada una cadena de texto `frase`, devuelve `True` si es un palíndromo; en caso contrario, devuelve `False`. Un palíndromo es una cadena que se lee igual de izquierda a derecha que de derecha a izquierda. Además, no distingue entre mayúsculas y minúsculas, y ignora todos los caracteres que no sean letras ni números.

El método `.isalnum()` en Python se utiliza para verificar si todos los caracteres de una cadena son alfanuméricos, es decir:

- Letras (mayúsculas o minúsculas): A-Z, a-z
- Números: 0-9

[Volver al contenido](#)

✓ Ejercicio 1.9: Estructuras de datos

Escribe una función en Python para obtener la intersección de dos listas.

Por ejemplo, si:

```
A = [1, 2, 3, 4, 5]
B = [0, 1, 3, 7]
```

Entonces la salida debe ser:

```
[1, 3]
```

```
def interseccion_listas(lista_1, lista_2):
    interseccion = set(lista_1) & set(lista_2)
    return list(interseccion)
```

```
A = [1, 2, 3, 4, 5]
B = [0, 1, 3, 7]
interseccion_listas(A,B)
```

```
[1, 3]
```

```
interseccion_listas = (lambda lista_1, lista_2: list(set(lista_1) & set(lista_2)))
print(interseccion_listas)
```

`[1, 3]`[Volver al contenido](#)

✓ Ejercicio 1.10: Estructuras de control

Escribe una función que evalúe si una persona cumple con los requisitos para acceder a un crédito, considerando su edad, nivel de ingresos y monto de deuda. La función debe devolver un mensaje con el veredicto final, además los datos deberán ser solicitados directamente al usuario a través de la consola función `input()`.

Para que una persona sea considerada apta para recibir el crédito, debe cumplir con todos los siguientes requisitos:

- Edad mínima: Tener 21 años o más.
- Ingresos mensuales: Percibir más de \$15,000.
- Relación deuda/ingresos: El porcentaje de deuda respecto al ingreso debe ser menor a 40%.

```
def evaluar_cliente():  
    try:  
        nombre = input("Ingresa tu nombre: ")  
        edad = int(input("Ingresa tu edad: "))  
        ingresos = float(input("¿Cuál es tu ingreso mensual? "))  
        deuda = float(input("¿A cuánto asciende tu deuda?"))  
  
        if edad >= 21 and ingresos >= 15000 and deuda / ingresos < 0.4:  
            mensaje = "Eres candidato a un crédito"  
        else:  
            mensaje = "Crédito denegado"  
    except ValueError:  
        mensaje = "La edad, el ingreso y la deuda deben de ser valores numéricos"  
    return mensaje
```

```
evaluar_cliente()
```

```
Ingresa tu nombre: Héctor  
Ingresa tu edad: Veinte  
'La edad, el ingreso y la deuda deben de ser valores numéricos'
```

[Volver al contenido](#)

Ejercicio 1.11: Numpy

Crea un array NumPy de temperaturas diarias (30 elementos).

a) Calcula: media, máximo, mínimo.

b) Aplica la estadarización Z-score.

c) Cuenta los días con temperatura superior a la media.

```
import numpy as np
```

```
# Crear un array con 30 elementos
temperaturas = np.random.uniform(low=20, high=35, size=30)
temperaturas
```

```
array([30.58051398, 22.05527851, 29.7546139 , 31.1591154 , 28.57526575,
       33.36460486, 24.23405651, 28.35398422, 28.33224379, 31.19908873,
       28.40026648, 33.99891713, 31.66888514, 22.26136749, 26.249695 ,
       31.23119092, 25.93345538, 32.95504336, 22.19865715, 31.9625254 ,
       32.91413486, 31.71745486, 22.35906872, 24.49938937, 34.58914451,
       27.05365414, 26.99054891, 34.78647941, 33.9757588 , 29.13466741])
```

```
# Obtener media, minimo, máximo, desviación estándar
media = temperaturas.mean()
min = temperaturas.min()
max = temperaturas.max()
de = temperaturas.std()
media, min, max
```

```
(np.float64(29.082969002911742),
 np.float64(22.055278508056166),
 np.float64(34.786479406013775))
```

```
# Transformar los datos
z_score = (temperaturas-media)/de
z_score
```

```
array([ 0.38315274, -1.7980621 ,  0.17184297,  0.53119018, -0.12989786,
        1.09547328, -1.24061323, -0.18651361, -0.19207598,  0.54141751,
       -0.17467211,  1.25776456,  0.66161676, -1.74533343, -0.72490423,
        0.54963098, -0.80581538,  0.99068537, -1.76137811,  0.73674577,
        0.98021877,  0.67404351, -1.72033619, -1.17272678,  1.40877654,
       -0.51920815, -0.53535387,  1.45926544,  1.25183941,  0.01322724])
```

```
# Días con temperatura superior a la media
temperaturas[temperaturas>media].shape[0]
```

16

[Volver al contenido](#)

✓ Ejercicio 1.12 Pandas

Dispones de la información del inventario y las ventas de una tienda. Con base en estos datos, elabora un resumen que incluya: qué productos se agotaron, cuáles se vendieron más y cuáles no se vendieron.

```
import pandas as pd

inventario = {
    "producto": [
        "Agua embotellada", "Refresco de cola", "Jugo de mango",
        "Sopa instantánea", "Sandwich de jamón", "Papas fritas",
        "Pan integral", "Pan dulce", "Galletas",
        "Shampoo en sobre", "Desodorante en barra", "Toallitas húmedas"
    ],
    "categoria": [
        "Bebidas", "Bebidas", "Bebidas",
        "Alimentos preparados", "Alimentos preparados", "Botanas",
        "Panadería", "Panadería", "Panadería",
        "Higiene personal", "Higiene personal", "Higiene personal"
    ],
    "inventario_inicial": [4, 20, 18, 15, 12, 20, 30, 25, 22, 40, 35, 50],
}

ventas = {
    "producto": [
        "Agua embotellada", "Refresco de cola", "Jugo de mango",
        "Sopa instantánea", "Papas fritas",
        "Pan integral", "Pan dulce", "Galletas",
        "Shampoo en sobre", "Desodorante en barra", "Toallitas húmedas",
        "Agua embotellada", "Papas fritas", "Toallitas húmedas"
    ],
    "ventas": [2, 1, 3, 2, 2, 1, 1, 1, 4, 1, 3, 2, 1,1],
}

df_inventario = pd.DataFrame(inventario)
df_ventas = pd.DataFrame(ventas)
```

```
df_inventario
```

	producto	categoria	inventario_inicial
0	Agua embotellada	Bebidas	4
1	Refresco de cola	Bebidas	20
2	Jugo de mango	Bebidas	18
3	Sopa instantánea	Alimentos preparados	15
4	Sandwich de jamón	Alimentos preparados	12
5	Papas fritas	Botanas	20
6	Pan integral	Panadería	30
7	Pan dulce	Panadería	25
8	Galletas	Panadería	22
9	Shampoo en sobre	Higiene personal	40
10	Desodorante en barra	Higiene personal	35
11	Toallitas húmedas	Higiene personal	50

```
df_ventas_totales = df_ventas.groupby('producto')['ventas'].sum().to_frame()
df_ventas_totales
```

	ventas
producto	
Agua embotellada	4
Desodorante en barra	1
Galletas	1
Jugo de mango	3
Pan dulce	1
Pan integral	1
Papas fritas	3
Refresco de cola	1
Shampoo en sobre	4
Sopa instantánea	2
Toallitas húmedas	4

```
df = pd.merge(df_inventario, df_ventas_totales, how='left', on='producto')
df
```

	producto	categoria	inventario_inicial	ventas
0	Agua embotellada	Bebidas	4	4.0
1	Refresco de cola	Bebidas	20	1.0
2	Jugo de mango	Bebidas	18	3.0
3	Sopa instantánea	Alimentos preparados	15	2.0
4	Sandwich de jamón	Alimentos preparados	12	NaN
5	Papas fritas	Botanas	20	3.0
6	Pan integral	Panadería	30	1.0
7	Pan dulce	Panadería	25	1.0
8	Galletas	Panadería	22	1.0
9	Shampoo en sobre	Higiene personal	40	4.0
10	Desodorante en barra	Higiene personal	35	1.0
11	Toallitas húmedas	Higiene personal	50	4.0

```
df['ventas'] = df['ventas'].fillna(0)
df
```

	producto	categoria	inventario_inicial	ventas
0	Agua embotellada	Bebidas	4	4.0
1	Refresco de cola	Bebidas	20	1.0
2	Jugo de mango	Bebidas	18	3.0
3	Sopa instantánea	Alimentos preparados	15	2.0
4	Sandwich de jamón	Alimentos preparados	12	0.0
5	Papas fritas	Botanas	20	3.0
6	Pan integral	Panadería	30	1.0
7	Pan dulce	Panadería	25	1.0
8	Galletas	Panadería	22	1.0
9	Shampoo en sobre	Higiene personal	40	4.0
10	Desodorante en barra	Higiene personal	35	1.0
11	Toallitas húmedas	Higiene personal	50	4.0

```
df['inventario_final'] = df['inventario_inicial'] - df['ventas']
df
```

	producto	categoria	inventario_inicial	ventas	inventario_final
0	Agua embotellada	Bebidas	4	4.0	0.0
1	Refresco de cola	Bebidas	20	1.0	19.0
2	Jugo de mango	Bebidas	18	3.0	15.0
3	Sopa instantánea	Alimentos preparados	15	2.0	13.0
4	Sandwich de jamón	Alimentos preparados	12	0.0	12.0
5	Papas fritas	Botanas	20	3.0	17.0
6	Pan integral	Panadería	30	1.0	29.0
7	Pan dulce	Panadería	25	1.0	24.0
8	Galletas	Panadería	22	1.0	21.0
9	Shampoo en sobre	Higiene personal	40	4.0	36.0

```
# qué productos se agotaron
productos_agotados = df[df['inventario_final']==0]['producto'].to_list()
```

```
['Agua embotellada']
```

```
# cuáles se vendieron más
df.sort_values(by='ventas', ascending=False).head(5).reset_index()[['producto',
```

	producto	ventas
0	Agua embotellada	4.0
1	Toallitas húmedas	4.0
2	Shampoo en sobre	4.0
3	Jugo de mango	3.0
4	Papas fritas	3.0

```
# cuáles no se vendieron
productos_no_vendidos = df[df['ventas']==0]['producto'].to_list()
productos_no_vendidos
```

```
['Sandwich de jamón']
```

[Volver al contenido](#)

✓ Ejercicio 1.13 Matplotlib

Visita la página <https://python-graph-gallery.com/> y elabora una gráfica de dispersión con los precios de cierre de las acciones de Pepsi y Coca-cola. Utiliza el archivo `acciones_precios_cierre.csv` disponible en *Piazza*.

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import pandas as pd
precios_acciones = pd.read_csv("/content/drive/MyDrive/Diplomado/Grupo 42/Teori
```

precios_acciones

	Fecha	Coca	Pepsi
0	03/01/2023	62.950001	179.410004
1	04/01/2023	62.919998	178.970001
2	05/01/2023	62.200001	177.100006
3	06/01/2023	63.400002	181.100006
4	09/01/2023	62.610001	179.330002
...	...	...	...
163	28/08/2023	60.560001	180.250000
164	29/08/2023	60.500000	180.929993
165	30/08/2023	60.470001	181.080002
166	31/08/2023	59.830002	177.919998
167	01/09/2023	59.310001	175.320007

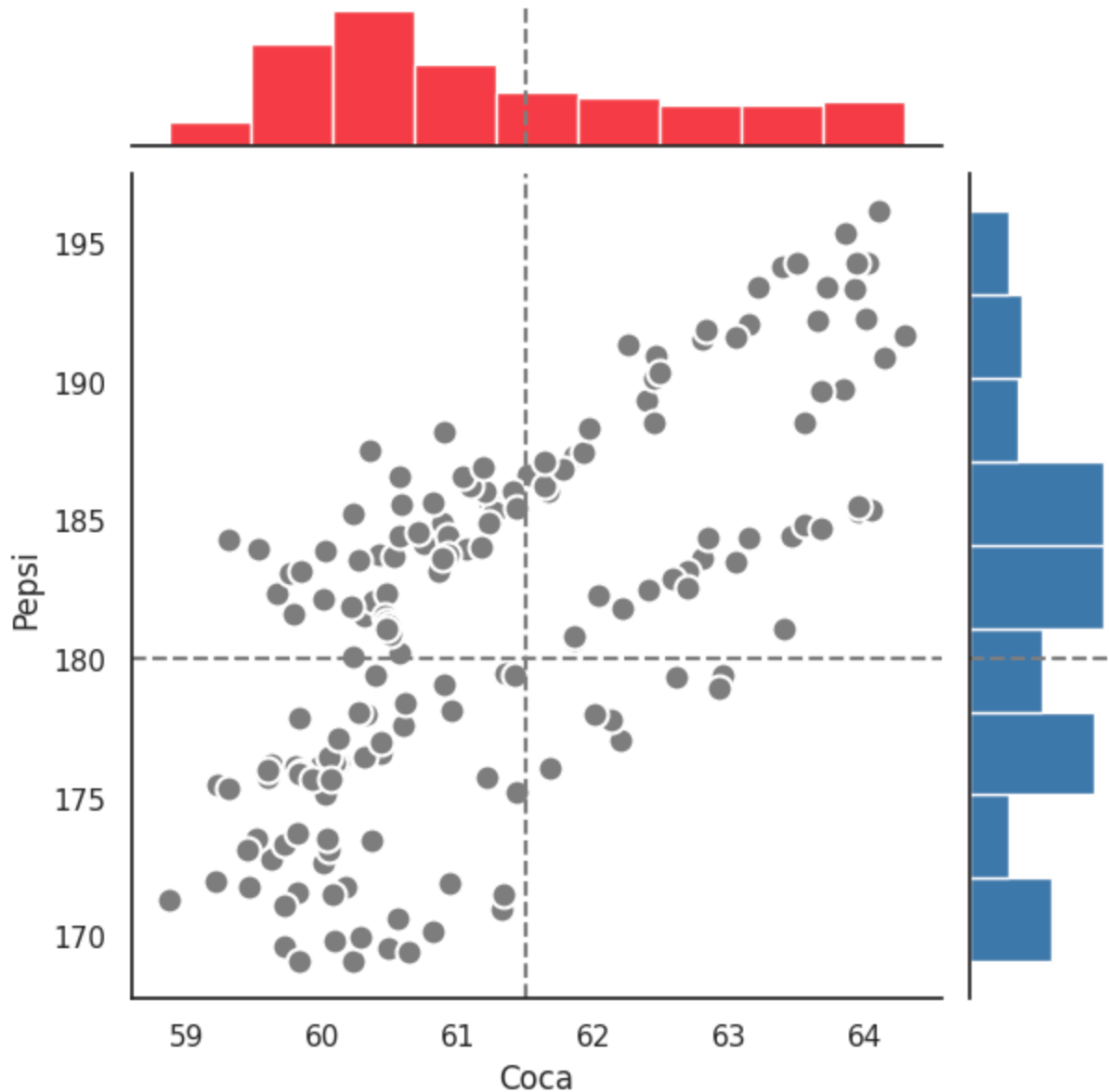
168 rows × 3 columns

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```

g = sns.JointGrid()
x, y = precios_acciones["Coca"], precios_acciones["Pepsi"]
sns.scatterplot(x=x, y=y, color="gray", s=100, linewidth=1.5, ax=g.ax_joint)
sns.histplot(x=x, color="#F40009", ax=g.ax_marg_x)
sns.histplot(y=y, color="#004B93", ax=g.ax_marg_y)
g.refline(x=61.5, y=180)
plt.show()

```



[Volver al contenido](#)

✓ Ejercicio 1.14: SQL

Realiza las siguientes consultas para la tabla `t_customers` de la base de datos `customers` disponible en *Piazza*.

- a) Seleccione todos los registros en los que la columna Ciudad tenga el valor Berlín y la columna Código postal tenga el valor 12209.
- b) Seleccione todos los registros en los que la columna País tenga el valor Alemania y la columna Ciudad no tenga el valor Berlín.
- c) Seleccione todos los registros cuya columna Ciudad tenga el valor Madrid o Barcelona.

[Volver al contenido](#)